

Retrieval-Augmented Generation (RAG)

Abdul Wasay

What RAG Is

A large language model only knows what was in its training data, and that knowledge stops at a fixed cutoff date. RAG fixes this by giving the model access to an external knowledge source at answer time. Instead of relying purely on what the model memorized, the system retrieves relevant text from a document store and hands it to the model as context before generating a response. The model then answers using both its own reasoning and the retrieved facts, rather than guessing from memory alone.

RAG has two separate pipelines: ingestion, which happens once (or whenever documents change), and query, which happens every time a user asks a question. Ingestion builds the knowledge base; query uses it.

Ingestion Pipeline

Documents are prepared and indexed ahead of time so they can be searched quickly later.

Ingestion Pipeline

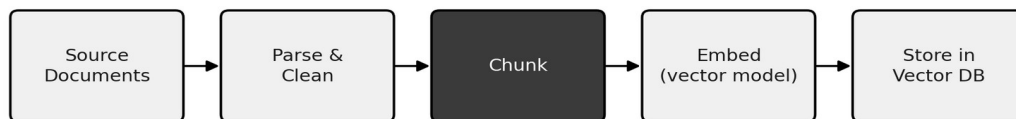


Figure 1: Documents are cleaned, split into chunks, embedded, and stored as vectors.

Query Pipeline

When a user asks a question, the system searches the same vector store for the most relevant chunks and feeds them to the LLM.

Query Pipeline

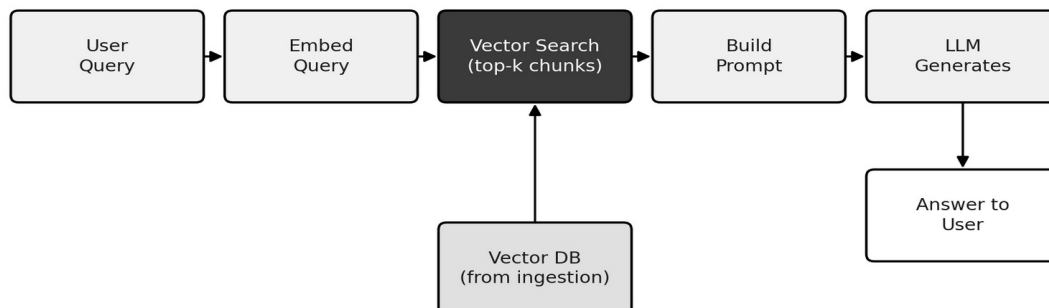


Figure 2: The query is embedded, matched against stored vectors, and the retrieved chunks are inserted into the prompt sent to the LLM.

Chunking

Chunking is the process of splitting long documents into smaller pieces before embedding them. Embedding models and vector search work on fixed-size pieces of text, not entire documents, so a 50-page manual has to be broken into chunks (typically a few hundred tokens each) before it can be indexed. Each chunk gets its own vector embedding and is retrieved independently.

Why chunk size matters

Chunk size controls the trade-off between precision and context. Too small, and a chunk loses the surrounding information needed to make sense of it: a sentence like "it costs \$200 more" is useless without the paragraph explaining what "it" is. Too large, and each chunk mixes multiple topics, which dilutes the vector embedding and makes retrieval less precise, while also wasting space in the LLM's context window on irrelevant text.

Aspect	Chunks too small	Chunks too large
Context per chunk	Loses surrounding meaning; facts get split from their subject	Keeps context intact
Retrieval precision	More chunks match the query, but many are noise	Fewer, broader matches; relevant text buried in irrelevant text
Prompt cost	Need more chunks (higher k) to cover a topic	Each chunk consumes more of the context window
Answer quality	Fragmented, disjointed answers	Diluted relevance; LLM must filter out noise

Where RAG Helps a Business Like Petalnex

For a growing business with scattered documentation, product data, and repetitive customer questions, RAG turns existing content into a queryable knowledge base without retraining a model.

- **Repetitive support inquiries:** Customers and staff ask the same questions repeatedly (order status, return policy, product care instructions), and answers currently live scattered across PDFs, spreadsheets, and email threads. A RAG-powered assistant can answer directly from that existing content instead of someone searching for it manually.
- **Slow internal knowledge lookup:** New employees have to read through manuals, SOPs, and old tickets to find out how things work. A RAG system over internal documentation lets them ask a question in plain language and get an answer sourced from the actual company documents, cutting onboarding time.
- **Outdated static FAQs and chatbot scripts:** Product specs, pricing, and availability change often, and static FAQ pages or chatbots go stale. RAG retrieves from the live document store at query time, so as long as the source documents are updated, answers stay current without retraining or manually rewriting scripted responses.